

PYTHON PROGRAMMING LAB

PROJECT REPORT

PYTHON CALCULATOR

A Command-Line Scientific Calculator with History & Automated Testing

Submitted By

Anant

USN: CD25002

B.Tech CSE (Data Science)

Course: Python Programming Lab

S.B. Jain Institute of Technology, Management & Research, Nagpur

Department of Computer Science & Engineering (Data Science)

1. Problem Statement

Basic calculators are useful for performing simple arithmetic operations, but they typically do not provide advanced mathematical functions, a record of past calculations, or robust handling of invalid input. Users who need to repeat similar calculations, verify a previous result, or evaluate more complex expressions are often forced to switch between multiple tools.

The objective of this project is to design and implement a Python-based command-line calculator that performs both basic and advanced mathematical operations, evaluates arithmetic expressions, maintains a persistent calculation history, and handles invalid input gracefully without crashing or terminating unexpectedly.

2. Objectives

- To develop a fully interactive, menu-driven calculator application using Python.
- To implement basic arithmetic operations: addition, subtraction, multiplication, division, modulus, and floor division.
- To implement advanced operations: power, square root, absolute value, factorial, and percentage.
- To evaluate general mathematical expressions typed directly by the user.
- To maintain a calculation history that persists across sessions using a JSON file.
- To implement input validation and error handling for all operations (e.g. division by zero, negative square roots, non-numeric input).
- To perform automated testing of core calculator logic using Python's **unittest** framework.
- To practice modular programming and version control using Git and GitHub.

3. Features

Basic Operations

- Addition, Subtraction, Multiplication, Division
- Modulus (remainder)
- Floor Division

Advanced Operations

- Power (exponentiation)
- Square Root
- Absolute Value
- Factorial (for non-negative integers, including large values using arbitrary-precision integers)
- Percentage calculation

Other Features

- Direct evaluation of arithmetic expressions typed by the user
- Calculation history saved to and loaded from a JSON file, so past results are not lost between runs
- Input validation that rejects invalid numbers, invalid menu choices, and undefined operations (e.g. division/modulus by zero) with a clear error message instead of crashing
- A clean, menu-driven text interface for navigating between operation categories

4. Technologies Used

Technology	Purpose
Python 3	Core programming language for the entire application
math	Built-in module used for square root and other mathematical functions
operator	Used to map menu choices to arithmetic operator functions
json	Used to persist calculation history between program runs
ast	Used for safely parsing and evaluating user-entered expressions
unittest	Python's built-in framework used for automated testing of calculator logic
Git	Version control for tracking changes to the codebase
GitHub	Remote repository hosting and project backup/collaboration
VS Code	Primary code editor used for development

5. Project Structure

The project follows a modular structure, separating the entry point, core calculation logic, history management, utility functions, and automated tests into individual files.

```
Calculator/
■■■ main.py           # Entry point; displays the menu and drives the program loop
■■■ calculator.py     # Core arithmetic and advanced-operation functions
■■■ history.py        # Reads/writes calculation history to a JSON file
■■■ utils.py          # Input validation and helper/utility functions
■■■ Tests/            # Unit tests for calculator logic (unittest)
■■■ README.md         # Project documentation
■■■ .gitignore
■■■ requirements.txt
■■■ Report/           # This project report and supporting screenshots
```

6. Working / Implementation

On launch, **main.py** presents the user with a top-level menu. Selecting Basic Operations shows a submenu of addition, subtraction, multiplication, division, modulus, and floor division.

```
===== BASIC OPERATIONS =====
. Addition
. Subtraction
. Multiplication
. Division
. Modulus
. Floor Division
. Back
```

Figure 1: Basic Operations Menu

Selecting Advanced Operations shows a second submenu covering power, square root, absolute value, factorial, and percentage. The user is prompted for the required input(s) for the chosen operation.

```
1. Power
2. Square Root
3. Absolute Value
4. Factorial
5. Percentage
6. Back

Enter your choice: 4
Enter a non-negative integer: █
```

Figure 2: Advanced Operations Menu

For example, choosing option 4 (Factorial) and entering the non-negative integer 44 correctly computes and displays the full factorial value using Python's arbitrary-precision integers,

confirming that large results are handled without overflow.

```
Enter your choice: 4  
Enter a non-negative integer: 44  
Result = 2658271574788448768043625811014615890319638528000000000  
Press Enter to continue...
```

Figure 3: Factorial of 44 Computed Successfully

7. Error Handling

Robust error handling was a core design goal so that the program never terminates unexpectedly on invalid input. The following cases are explicitly handled:

- **Division / Modulus / Floor Division by zero** — caught and reported as a clear error message instead of raising an unhandled exception.
- **Non-numeric input** — caught using try/except around type conversion, prompting the user to re-enter a valid number.
- **Invalid menu choice** — out-of-range or non-numeric menu selections are rejected and the menu is redisplayed.
- **Negative input for Factorial / Square Root** — rejected with a message explaining that a non-negative value is required.
- **Malformed expressions** — expression evaluation is wrapped in error handling so an invalid expression does not crash the program.

8. Testing and Results

Core calculator functions were verified with automated unit tests written using Python's **unittest** framework, run via **unittest discover** against the Tests/ directory. All 12 tests passed successfully.

```
PS C:\Users\de11\OneDrive\Desktop\Python calculator> git checkout -b feature/improve-calculator
Switched to a new branch 'feature/improve-calculator'
PS C:\Users\de11\OneDrive\Desktop\Python calculator> python -m unittest discover -s Tests
.....
-----
Ran 12 tests in 0.001s

OK
PS C:\Users\de11\OneDrive\Desktop\Python calculator> |
```

Figure 4: All 12 Automated Unit Tests Passing

Manual Testing

Test Case	Result
Addition	15
Power	32
Division by zero	Handled — Error message shown
Expression evaluation	35

9. Git & GitHub Workflow

The project was version-controlled with Git from the start and pushed to a remote repository on GitHub, allowing incremental commits and a safe backup of the codebase.

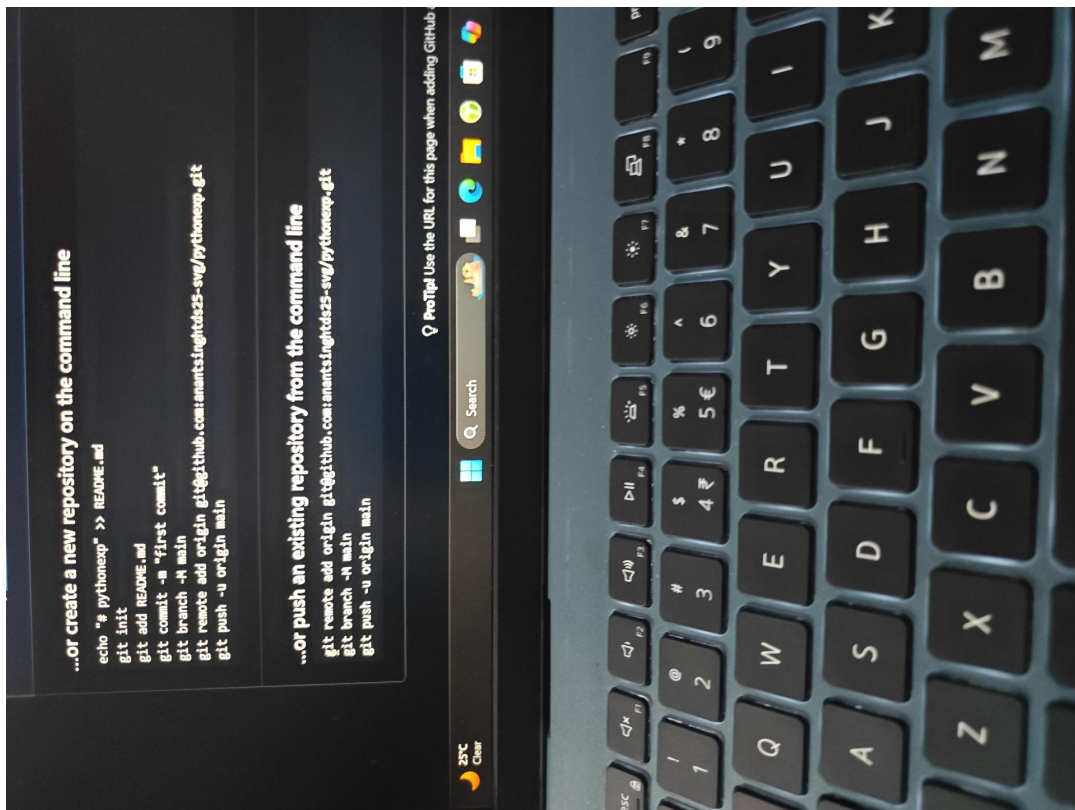


Figure 5: Initializing the Repository and Pushing to GitHub

Development of new functionality was done on a dedicated feature branch (**feature/improve-calculator**), keeping the main branch stable. After implementing changes, the full test suite was run on the branch before merging, confirming that all 12 tests still passed.

10. Conclusion

The Python Calculator project successfully demonstrates a modular, menu-driven command-line application capable of performing both basic and advanced mathematical operations, evaluating expressions, and persisting calculation history. Comprehensive input validation and error handling ensure the program remains stable under invalid or unexpected input, and an automated test suite of 12 unit tests confirms the correctness of the core logic. The project also provided practical experience with modular Python design and a Git/GitHub-based development workflow.

11. Future Scope

- Add a graphical user interface (GUI) using Tkinter or a web-based front end.
- Extend scientific functions to include trigonometric, logarithmic, and statistical operations.
- Add unit conversion (currency, length, weight, temperature) as an additional menu category.
- Store history in a lightweight database (e.g. SQLite) instead of a flat JSON file for larger, searchable histories.
- Package the application for distribution via pip / PyPI.
- Add a configuration option for angle units (degrees/radians) once trigonometric functions are added.

12. Project Repository

The complete source code, tests, and documentation for this project are available on GitHub:

<https://github.com/anantsinghtds25-svg/pythonexp>

Clone with:

```
git clone https://github.com/anantsinghtds25-svg/pythonexp.git
```